

Building an Intelligent Conversational Agent

Abdelmatine Dadah
a.daddah@student.vu.nl

Bennett Hampel
b.w.hampel@student.vu.nl
2862611

Kabir D'Souza
k.s.dsouza@student.vu.nl
2865270

Mikołaj Duchliński
m.duchlinski@student.vu.nl
2795928

Fadu Ashraf Eshak Abdou
fady.ashraf.eshak.abdou@student.vu.nl
2798005

Introduction

This project focuses on the development of a conversational recipe recommending agent that we have designed to assist its users in choosing recipes through natural language interaction. The structured task involves guiding users through a dialogue to get relevant preferences such as available ingredients, dietary restrictions, preferred cuisine, and preparation time. Based on this information, the system then generates connected recipe recommendations. Dialogue systems are often used in real-world scenarios, think about virtual assistants, customer support systems, and recommendation platforms, due to their efficiency and goal-driven nature. This makes them noticeably suitable for practical scenarios such as our recipe recommending system.

The primary goal of our project is to implement a functional and user-friendly conversational agent that can provide its users personal recipe recommendations. To achieve this goal, the system recognizes user intents and extracts relevant information from user input and simultaneously keeps contextual awareness throughout the entire dialogue. Another of our aims is to design a structured dialogue flow that guides users toward a suitable recipe in an efficient way and without unnecessary interaction steps. Additionally, the agent is programmed to deliver clear and relevant responses that improve the overall user experience. By focusing on both technical performance and usability, the project aims to show us how task-oriented conversational agents can effectively support real-world recommendation tasks.

How Does Your Conversational Agent Work?

2.1 Functionality

The goal of the conversational agent is to let users intuitively search, filter through, and explore recipes with the help of a bot. By simply asking questions and talking to the bot, recipes and meals can be found, instead of manually searching through websites or books. The agent supports multiple main use cases, the first of which being recipe discovery. Based on a few different filters of what the user desires, whether it is a specific country's cuisine, a

specific meal of the day, or certain preferred ingredients. Another function of the agent is providing details about each recipe, including but not limited to preparation time, ingredients, and step by step instructions. Finally, the bot is also able to guide the user through the cooking steps, which helps while actively cooking.

2.2 Conversational Flow

To start the agent, the user must greet the bot by saying hello. Once that is done, the user may ask for a recipe using any of the previously mentioned preferred filters. The intent, which is what the user wants to achieve, is determined by the agent. The slots, which are the key pieces of information, are then extracted from the input based on the user's intent. The agent then queries the knowledge base, which is where the recipes are stored in the Prolog programming language. The information from the knowledge base is then returned to the user in the form of bullet points and steps, and the conversation can continue from there.

2.3 *start.html*

The only available pattern is "start" and the only available interaction is clicking the start button.

2.4 *welcome.html*

The only available pattern is "c10 pattern" which will trigger the agent to self identify (as a greeting action) and continue to the next pattern in the agenda, only if the user responds with input classified as greeting.

All other input from the user will be recognized as pattern b12 (not matching the classification of database) or pattern b13 (out of context), and trigger a corresponding response from the agent, guiding the user towards the issue in his/her input.

2.5 *recipe_overview.html*

The available patterns are a50recipeSelect (requesting a specific recipe or asking for a recommendation) and the feature management patterns: a21featureRequest (adding filters), a21removeKeyFromMemory (removing filters), and a21noMoreFilters (finalizing the search).

A21featureRequest and a21removeKeyFromMemory are both valid on the recipe_overview.html website and for the smoothness of interaction, recipes will also be shown on it if applied filters narrow down recipes counter to 15 or below. This way there are no unnecessary website transitions and data loading. A21noMoreFilters can be triggered any time on this website, and if the number of current recipes available is 100 or below, then pictureGranted will succeed, and the agent will move the user to recipe_overview2.html where all these recipes can be reviewed.

All other input from the user will be recognized as pattern b12 (not matching the classification of database) or pattern b13 (out of context), and trigger a corresponding response from the agent, guiding the user towards the issue in his/her input.

2.6 *recipe_overview2.html*

This website is only being loaded if a21noMoreFilters rule succeeds within which the pictureGranted predicate has to succeed. This will only happen if the current recipe counter is between 16 and 100 recipes available and the user requests to see results of filtering.

The available patterns are again a50recipeSelect (requesting specific recipe or asking for a recommendation) and the same feature management patterns: a21featureRequest and a21removeKeyFromMemory.

All other input from the user will be recognized as pattern b12 (not matching the classification of database) or pattern b13 (out of context), and trigger a corresponding response from the agent, guiding the user towards the issue in his/her input.

2.7 *recipe_confirmation.html*

The available patterns is a50recipeConfirm with confirmation, appreciation and disconfirmation varieties. Both confirmation and appreciation will insert the c43 pattern and therefore move to the closing.html website, while disconfirmation will insert a50recipeSelect and move back to recipe_overview.html website.

All other input from the user will be recognized as pattern b12 (not matching the classification of database) or pattern b13 (out of context), and trigger a corresponding response from the agent, guiding the user towards the issue in his/her input.

2.8 *closing.html*

The available pattern is c43. The agent outputs action farewell, and the user can respond with a farewell (terminating the interaction) or a restart command (triggering the **restart** intent to reset the agent).

All other input from the user will be recognized as pattern b12 (not matching the classification of database) or pattern b13 (out of context), and trigger a corresponding response from the agent, guiding the user towards the issue in his/her input.

Intent and Slot Classifier

3.1 Intents and Slots

Intent classification and slot filling formed the core of the pipeline the agent understands. Intent classification identifies the user's goal (e.g. 'request_recipe' when the user asks for a recipe), while slotfilling extracts key parameters (e.g. cuisine type, dietary restrictions) from the expression. For example, in 'Find me a Mexican vegetarian recipe', intent classification should detect a 'recipe request' intent, and slot filling should identify 'Mexican' as cuisine and 'vegetarian' as a dietary restriction slot. Accurate intent/slot classification enables the agent to map user queries to associated actions and query the recipe database correctly. Slot-filling specifically is a vital task in task-oriented conversational systems, since extracting

slot values allows the agent to generate relevant responses and satisfy the requests of the user.

3.2 Architecture Summary

We use a BERT-based joint model with a shared Transformer encoder. A fine tuned BERT provides broad embedded context. After introducing each input utterance into BERT, the refined model applies two classifiers: a token-level slot classifier on each word's hidden state and a sentence-level intent classifier on the final hidden state of the special [CLS] token. In reality, one of the predefined intent labels is predicted by passing the [CLS] embedding through a softmax layer, and BIO slot tags are predicted by passing the remaining token embeddings through a sequence tagger (softmax). This joint architecture benefits from shared contextual encoding: the self-attention in BERT allows slot predictions to inform intent decisions and vice versa. Previous work had shown that fine-tuning BERT in this way results in state-of-the-art intent accuracy and slot-filling F1 on benchmark datasets.

As mentioned, model performance is evaluated using standard classification metrics, including accuracy, precision, recall and F1-score. Accuracy measures the proportion of correct predictions overall. While intuitive, it can be misleading when using wrong settings, as high accuracy may result from favoring majority classes. Precision reflects how often predicted intents or slots are correct, reducing false positives and preventing system errors. Recall measures how many relevant intents or slot values are correct. F1-score is the harmonic mean of precision and recall, balancing these two measures and is therefore our primary used metric when comparing.

In the following paragraphs we use Figures 1 and 2, see Appendix.

In the original model (Figure 1) the intent classifier reached a F1-score of about 89%, which means that the model correctly identifies user intents in most cases, but has some confusion between semantically similar intents. On the contrary, the slot-filling component reached a higher F1-score of roughly 98%, demonstrating a rock solid token-level performance. This shows that the model is very effective at extracting relevant slot values, such as cuisine type, dietary restrictions, and ingredients once the intent is known. In practice, this means that even when there is intent ambiguity, the extracted parameters are still accurate and usable for recipe retrieval.

To improve performance, we have fine-tuned our model using the ADAM optimizer with a Softmax output function for the intent classification and slot tagging. We have adjusted the following hyperparameters: the number of epochs was increased to 5, the batch size was increased to 32, and the learning rate was set to $1e-4$. After fine-tuning (Figure 2), intent classification performance had improved to around 92% accuracy, resulting in much better disambiguation between close related intents and less classification errors. Slot-filling performance slightly improved, reaching an F1-score of approximately 98.6%. Although the measurable gain is small, this improvement is still meaningful given the already high baseline and tells us that there is increased robustness in handling overlapping or ambiguous slot values.

3.3 Analysis of Challenges

Despite strong overall performance, several common errors have occurred. Overlap errors arise when slot spans are wrongly aligned or overlap in meaning. For example, 'chocolate cake' might be tagged as one span for dessert, but the model might split it incorrectly. Ambiguous intent errors happen when similar phrases could imply different intents. 'Add filter' could be interpreted as the `addFilter` intent or as providing a `filterType` slot value. Such ambiguity can lead to the model assigning the wrong intent label. Slot misclassifications occur when tokens are assigned the wrong slot type. For example, 'gluten-free' might be mislabeled as an ingredient instead of a dietary restriction. These correspond to general 'Wrong Slot' errors. In practice, we observed that some slot types and nested phrases can trigger these small mistakes.

3.4 Improvement and Justification

Adjusting the training configuration resulted in measurable gains. Increasing the number of epochs and batch size allowed the model more training iterations per epoch and more stable gradient estimates, while a higher learning rate sped up convergence on our dataset. Together, these changes raised intent accuracy by a few points and slot F1 some tenths of a point (see the figures). Fine-tuning BERT allowed the model to leverage contextual information across entire utterances, improving disambiguation between similar intents and slot labels. After fine-tuning and connecting the entire pipeline with Google STT and our custom NLU, the functionality of the MARBEL agent was very accurate, where all example scenarios and queries were classified correctly, hence the reason the agent progressed with patterns accordingly.

4. Exclusion Mechanism

4.1 Exclusion approach

For our exclusion mechanism, we decided on using a slot-based approach for our NLU. With this approach, extensions are feasible; new concepts or slots can be introduced by adding new slots to both the ontology and the data with examples used for model training.

The original ontology supported main culinary concepts such as ingredients, cuisines, and dietary restrictions, which is sufficient for a basic agent providing help and filtering based on what the user wants in recipes without removing or excluding items from the search. An important change that was made is the separation between the intent of inclusions and exclusions through the training of the agent. This was first achieved by adding the same ingredient slots used for inclusion to exclusion slots for ingredients under the **excludeIngredient** slot in the ontology, adding the same values of cuisine slots under **excludeCuisine**, and adding tags to the ontology. The addition of this exact data to the ontology allows the NLU model to catch all possible values for slots that inclusion slots do, all of which are included in the **recipe_database.pl** file. These changes ensure that inclusion and exclusion constraints are semantically distinct because of slot keys while

remaining consistent with the existing ontology. Importantly, the introduction of these slots did not disrupt the system, as the slot-based design allows the model to learn new labels through data examples alone.

At a later stage, the same mechanism was applied to add **excludeDietaryRestriction** and **excludeMealType** slots to complete a full exclusion implementation. Again, this was done by adding the corresponding inclusion slot values to the ontology for the new exclusion slots. All changes mentioned thus far can be found in the **ontology.json** file from line 6818 until line 12405.

After adding ontology values for exclusion keys, training data for these specific combinations of keys and values had to be added. No additional intents were required, as the model can reuse the **addFilter** intent effectively for adding exclusions, while the **deleteFilterValue** intent can handle removing filters. We do not need to train the model specifically to remove the exclusion slots, as the specific filter slot and name are handled by the MARBEL agent. Therefore, all added examples use the **addFilter** intent. We have generated and added exclusion test and train examples; an example is provided below:

```
{
  "id": "exing005",
  "text": "Exclude cheese from the recipe.",
  "intent": "addFilter",
  "slots": { "excludeIngredient": "cheese" }
},
```

All train data examples for exclusion slots can be seen in the **exclusion_example.json** file, and test data examples can be seen in the **exclusion_examples_test.json** file. These two files were part of the data folder of the model and were loaded with instances for the model to train with. The training slot distribution is visualized in the tables within the appendix (see Figure 6. Merged Slot Type Classification Report in Appendix). We have added a corresponding number of train examples - **excludeCuisine** (108), **excludeIngredient** (282), **excludeMealType** (151), **excludeDietaryRestriction** (251), and **tag** (89) - to prioritize and balance the data we want the model to learn. Different test examples were also added for all

INTENT DISTRIBUTION (Capped Gradient)				SLOT DISTRIBUTION (Original Gradient)			
	Train	Test	Total		Train	Test	Total
addFilter	3638	1367	5005	ingredient	936	296	1232
deleteFilterValue	564	176	740	cuisine	722	198	920
recipeRequest	359	113	472	dietaryRestriction	565	175	740
requestRecommendation	278	100	378	duration	477	222	699
disconfirmation	272	101	373	recipe	354	111	465
confirmation	241	119	360	excludeIngredient	282	100	382
fallback	182	100	282	excludeDietaryRestriction	251	80	331
appreciation	180	100	280	mealType	228	146	374
deleteParameter	154	104	258	easyKeyWord	188	114	302
checkCapability	151	101	252	ingredientType	156	51	207
greeting	144	100	244	excludeMealType	151	60	211
restart	123	100	223	filterType	141	102	243
farewell	119	100	219	excludeCuisine	108	101	209
noMoreFilters	109	100	209	tag	89	27	116

these exclusion slots.

The specific numbers of examples were somewhat random at first, as two people worked separately on them, leading to some categories having 100–150 examples and others around 250. Still, after training, the model worked well and passed the required thresholds. Therefore, we decided not to update the number of examples and kept the well-working model.

The model was trained in 10 epochs, with a 2e-05 learning rate and a batch size of 16. These hyperparameters resulted in a great intent accuracy of 93.80% (see Figure 4. Intent Classification Report in Appendix) and slot accuracy of 98.70% (see Figure 5. BIO Slot Classification Report in Appendix), providing excellent functionality within the framework. All patterns and queries were tested and functioned correctly. The exclusion mechanism was tested heavily, and thanks to the robust implementation of the MARBEL agent, all exclusion queries worked flawlessly.

4.2 MARBEL Agent Implementation

The implementation of exclusion did not differ significantly from recipe selection via inclusion filtering. In the **recipe_selection.pl** file, we added all exclusion slots with the **addFilter** intent as **applyFilter** rules using corresponding slots as atoms. For example, an **addFilter** intent with an **excludeCuisine** slot classified by the NLU will match the following rule:

```
applyFilter('excludecuisine', Cuisine, RecipeIDsIn, RecipeIDsOut) :-  
    findall(RecipeID, (member(RecipeID, RecipeIDsIn), not(cuisine(RecipeID,  
Cuisine)))), RecipeIDsOut).
```

This rule uses the **findall** predicate and includes the **not(cuisine(RecipeID, Cuisine))** negation to exclude the classified cuisine. In contrast, the inclusion rule would be:

```
applyFilter('cuisine', Cuisine, RecipeIDsIn, RecipeIDsOut) :-  
    findall(RecipeID, (member(RecipeID, RecipeIDsIn), cuisine(RecipeID, Cuisine)),  
RecipeIDsOut).
```

Similarly, for other exclusions in **recipe_selection.pl**:

- **excludeingredient**: uses **not(hasIngredient(R, NormalizedIngredient))**
- **excludedietaryrestriction**: uses **not(diet(R, Diet))**
- **excludemealtype**: uses **not(mealType(R, Type))**
- **excludetag**: uses **not(tag(R, Tag))**

Additionally, all exclusion rules require special formatting and atom retrieval. This functionality is primarily implemented in the **dialogflow.pl** file. In that file, **simplify/3** and **normalize_key/2** predicates were added for all exclusion slots, ensuring these slots are retrieved and passed to the memory predicates in the correct form. Corresponding templates for website content were also created, such as **['excludeingredient', "do not include ~a"]**.

Essentially, the exclusion implementation in MARBEL required negation in filtering rules and corresponding formatting predicates. However, exclusion filters were added as normal filters to the memory, keeping the implementation straightforward. Consequently, the memory

could receive a "with salt" filter (678 recipes) and a "without salt" filter, which correctly negates the salt ingredient ($898 - 678 \Rightarrow 220$ recipes).

4.3 Strength and Limitation

The main advantage of the chosen exclusion approach is the easy interpretation of negative constraints, as the excluded slots are simple and explicit. By modeling exclusions through dedicated slots (initially `excludeIngredient` and `excludeCuisine`, followed by `excludeDietaryRestriction` and `excludeMealType`), an explicit difference is created between positive user inputs (inclusion) and negative inputs (exclusion). This approach is also highly extensible; new exclusion categories can be included simply by adding them to the ontology and providing adequate training examples without needing to modify the system architecture.

On the contrary, the slot-based approach has its limitations. As the approach relies heavily on the explicit slot labels, the performance would be heavily dependent on how diversified the training data is. Slots that are not frequent or underrepresented in the training data may not be reliably detected by the model after being trained. It could also ignore such slots during the training process. This approach operates on a lot level and not a logical or semantic level, meaning the model cannot perform complex reasoning over what to exclude if a user used an input the model is not familiar with. However, such complexity of course requires much bigger models and different architecture than a simple BERT transformer. As a result, while this approach is robust and easy to scale up when needed to extend on it, it is best accompanied with an additional reasoning layer to support more meaningful exclusions.

Extensions to the Bot

5.1 NLU

Originally, the conversational pipeline relied on Dialogflow for intent recognition and entity extraction. However, to extend on the project and add further customization, the project pipeline was successfully connected with a custom model. Dialogflow was replaced with a Natural Language Understanding (NLU) component. This NLU is based on a BERT model trained for intent classification and slot filling, allowing the system to accurately interpret user requests and extract relevant information.

5.2 Visual Extensions

To improve the overall user experience and look of the system, the website was upgraded beyond the default interface provided by the Social Interaction Cloud framework. The desired outcome was a more sleek and modern look, which was achieved using ChatGPT as a quick design aid.

There was room for improvement with the recipe selection process, as the user had to choose a recipe by speech. This was at times difficult, as the names of the recipes were not always recognized by the bot. To make it easier and more streamlined for the user, an interactive recipe card feature was implemented. Once the list of matching recipes is small

enough, they appear on screen. To choose one, the user simply clicks on the desired meal. If, however, the user wishes to select a recipe by speech, this is still possible.

While textual instructions and ingredient lists provide essential information, visual demonstrations add another layer of user support. For this reason, a functional visual extension was introduced to help users during the cooking process: the integration of YouTube recipe videos. Placed alongside the chosen recipe, these videos provide users with a concrete reference for observing preparation methods, techniques, and timing as they cook.

This functionality was implemented through a server-side integration with the YouTube Data API. A dedicated endpoint was added to the Flask-based webserver used by the system. This endpoint takes the title of the selected recipe as input and performs a search query using the YouTube API. The first suitable video result returned by the API is then selected, and its unique video identifier is sent back to the frontend. The API key required for accessing the YouTube Data API is configured at the beginning of the web app file, as security is not an issue for this particular project.

On the frontend, a new visual section was added to the recipe confirmation page. This section is positioned directly beneath the recipe information. The video is displayed using an embedded YouTube player, which loads the video once its ID has been retrieved from the backend. (See figure in appendix).

Pilot User Study

6.1 Setup

The pilot user study was conducted with AI students who were classmates from the same course but belonged to a different project group. This participant selection ensured that users had relevant domain knowledge while remaining independent from the development of the system.

The evaluated system was a conversational agent integrated into a recipe recommendation website. The main purpose of the system was to recommend suitable recipes based on natural language user queries and applied filters.

Participants were asked to perform the following tasks:

- Ask the conversational agent for a recipe
- Apply specific filters when requesting a recipe
- Evaluate whether the recommended recipes matched their request

Data collection was performed through direct observation only. User interactions and behavior were observed while completing the tasks, without the use of surveys, interviews, or interaction logs.

6.2 Results from the Study

The pilot user study produced simple descriptive results based on observed user interactions. From a quantitative perspective, most users were able to successfully obtain relevant recipe recommendations. In some cases, the conversational agent required additional processing time before responding. Additionally, it was observed that ingredient types (meat) were sometimes not correctly recognized as excluded ingredients, but were instead interpreted as included ingredient types or filters, which occasionally led to less accurate recommendations.

From a qualitative perspective, users generally reported a positive experience when interacting with the system. Observations showed that users found the chatbot easy to use and effective in many situations.

6.3 Analysis

The pilot user study shows that the conversational agent performs well overall in recommending recipes that match user requests and applied filters. Most users were able to complete the assigned tasks successfully, indicating that the core functionality of the system is effective.

However, the study also revealed limitations in the Natural Language Understanding (NLU) component. In particular, ingredient types such as 'meat' were sometimes incorrectly interpreted as included ingredients or general filters instead of being recognized as excluded ingredients. This misinterpretation occasionally resulted in recommendations that did not fully align with the user's intent.

The main lesson learned from this pilot study is that improving intent and slot recognition in the NLU module would significantly enhance system accuracy and user satisfaction. These findings highlight the importance of refining ingredient exclusion handling before conducting a larger-scale user study or deploying the system in a real-world setting.

Conclusion

The goal of our project was to build an operating conversational agent that recommends recipes based on user input. This goal was realized, as our final designed system allowed users to interact successfully with a chatbot, apply filters, and receive relevant recipe recommendations through our website. Overall, the development process went excellent. Communication within the team was good, which helped with dividing tasks and combining the different parts of the system. For all team members, the project was also a chance to learn new tools and technologies, which helped improve teamwork and implementation.

There were no major technical issues that hindered the project, but testing showed some roadblocks. One common issue was with ASR (Automatic Speech Recognition), where the system sometimes produced some incorrect translations, which led to misunderstandings. In addition, due to limitations of the overall framework, the user had to repeat the input, and in some cases generated the same response twice. These issues did reduce the smoothness

of the interaction by a bit. However, these problems were likely caused by components of the EIS framework and were outside the scope of this project to fix.

Several improvements could be made in future work. Improving the ASR component and the NLU module would lead to more accurate and consistent system behavior. Expanding the recipe database would increase the usefulness of the system in real life. Furthermore, extending the user interface with additional features, such as a community where users can upload their own recipes, could improve the overall user experience.

Through this project, the team gained practical experience with NLU, MARBEL, and their integration within a large framework. The project also helped the team develop a clearer understanding of how different system components interact, as well as the challenges that were involved in building a complete conversational agent using existing frameworks.

APPENDIX

evaluation models (both models)

INCLUSION MODEL:

Figure 1. Original model's classification report (intent and slot performance). The intent classifier achieved ~89.3% accuracy (weighted F1 ~89%) and the slot-filler achieved ~98.3% F1 (98.4% accuracy) on the test set.

Intent Classification Report:				
	precision	recall	f1-score	support
addFilter	0.95	0.97	0.96	917
appreciation	0.90	0.98	0.94	100
checkCapability	0.94	0.97	0.96	101
confirmation	0.91	0.83	0.87	119
deleteFilterValue	0.81	0.87	0.84	176
deleteParameter	0.94	0.70	0.80	104
disconfirmation	0.59	0.93	0.73	101
fallback	0.72	0.71	0.71	100
farewell	0.97	0.94	0.95	100
greeting	0.90	0.96	0.93	100
noMoreFilters	0.90	0.36	0.51	100
recipeRequest	0.91	0.97	0.94	113
requestRecommendation	0.95	0.80	0.87	100
restart	0.86	0.89	0.88	100
accuracy			0.89	2331
macro avg	0.88	0.85	0.85	2331
weighted avg	0.90	0.89	0.89	2331
BIO Slot Classification Report:				
	precision	recall	f1-score	support
B-cuisine	0.90	0.95	0.92	174
B-dietaryRestriction	0.85	0.94	0.89	171
B-duration	0.84	0.72	0.77	204
B-easyKeyword	0.71	0.67	0.69	108
B-filterType	0.73	0.65	0.69	97
B-ingredient	0.85	0.93	0.89	266
B-mealType	0.77	0.67	0.72	140
B-recipe	0.91	0.84	0.87	111
I-cuisine	1.00	0.13	0.24	15
I-dietaryRestriction	0.91	0.92	0.91	251
I-duration	0.88	0.78	0.83	185
I-easyKeyword	0.68	0.66	0.67	87
I-filterType	0.00	0.00	0.00	16
I-ingredient	0.81	0.92	0.86	209
I-mealType	0.93	0.75	0.83	55
I-recipe	0.94	0.92	0.93	428
0	0.99	0.99	0.99	34779
accuracy			0.98	37296
macro avg	0.81	0.73	0.75	37296
weighted avg	0.98	0.98	0.98	37296
Merged Slot Type Classification Report:				
	precision	recall	f1-score	support
cuisine	0.92	0.90	0.91	189
dietaryRestriction	0.88	0.93	0.91	422
duration	0.88	0.77	0.82	389
easyKeyword	0.71	0.68	0.69	195
filterType	0.78	0.59	0.67	113
ingredient	0.84	0.94	0.89	475
mealType	0.84	0.71	0.77	195
recipe	0.94	0.91	0.93	539
micro avg	0.87	0.85	0.86	2517
macro avg	0.85	0.80	0.82	2517
weighted avg	0.87	0.85	0.86	2517

Figure 2. Our improved model's classification report (after fine-tuning)
Improved model's classification report (intent and slot performance). The intent classifier achieved ~92.0% accuracy (weighted F1 ≈92%) and the slot-filler achieved ~98.7% F1 (98.7% accuracy) on the test set.

Intent Classification Report:				
	precision	recall	f1-score	support
addFilter	0.97	0.97	0.97	917
appreciation	0.89	0.96	0.92	100
checkCapability	0.99	0.90	0.94	101
confirmation	0.85	0.87	0.86	119
deleteFilterValue	0.94	0.89	0.92	176
deleteParameter	0.92	0.91	0.92	104
disconfirmation	0.67	0.94	0.79	101
fallback	0.84	0.74	0.79	100
farewell	0.98	0.95	0.96	100
greeting	0.98	0.97	0.97	100
noMoreFilters	0.80	0.70	0.75	100
recipeRequest	0.93	0.96	0.94	113
requestRecommendation	0.86	0.85	0.85	100
restart	0.90	0.92	0.91	100
accuracy			0.92	2331
macro avg	0.90	0.89	0.89	2331
weighted avg	0.92	0.92	0.92	2331
BIO Slot Classification Report:				
	precision	recall	f1-score	support
B-cuisine	0.94	0.98	0.96	174
B-dietaryRestriction	0.96	0.89	0.92	171
B-duration	0.79	0.86	0.83	204
B-easyKeyword	0.85	0.74	0.79	108
B-filterType	0.82	0.82	0.82	97
B-ingredient	0.95	0.89	0.92	266
B-mealType	0.93	0.73	0.82	140
B-recipe	0.86	0.90	0.88	111
I-cuisine	1.00	0.73	0.85	15
I-dietaryRestriction	0.90	0.92	0.91	251
I-duration	0.84	0.85	0.85	185
I-easyKeyword	0.91	0.72	0.81	87
I-filterType	0.60	0.56	0.58	16
I-ingredient	0.93	0.84	0.88	209
I-mealType	0.95	0.95	0.95	55
I-recipe	0.95	0.87	0.91	428
0	0.99	1.00	0.99	34779
accuracy			0.99	37296
macro avg	0.89	0.84	0.86	37296
weighted avg	0.99	0.99	0.99	37296
Merged Slot Type Classification Report:				
	precision	recall	f1-score	support
cuisine	0.95	0.96	0.96	189
dietaryRestriction	0.93	0.92	0.93	422
duration	0.84	0.88	0.86	389
easyKeyword	0.88	0.73	0.80	195
filterType	0.80	0.80	0.80	113
ingredient	0.95	0.88	0.91	475
mealType	0.93	0.79	0.86	195
recipe	0.94	0.89	0.92	539
micro avg	0.91	0.87	0.89	2517
macro avg	0.90	0.86	0.88	2517
weighted avg	0.91	0.87	0.89	2517

Figure 3&4. Slot Distribution and Intent Distribution (Inclusion only)

SLOT DISTRIBUTION (Original Gradient)

	Train	Test	Total
ingredient	828	268	1096
cuisine	639	178	817
dietaryRestriction	557	172	729
duration	455	216	671
recipe	354	111	465
mealType	228	146	374
easyKeyword	187	114	301
filterType	141	102	243

INTENT DISTRIBUTION (Capped Gradient)

	Train	Test	Total
addFilter	2474	917	3391
deleteFilterValue	564	176	740
recipeRequest	359	113	472
requestRecommendation	278	100	378
disconfirmation	272	101	373
confirmation	241	119	360
fallback	182	100	282
appreciation	180	100	280
deleteParameter	154	104	258
checkCapability	151	101	252
greeting	144	100	244
restart	123	100	223
farewell	119	100	219
noMoreFilters	109	100	209

EXCLUSION MODEL:

```

RUN CONFIGURATION
Epochs: 10 | Batch Size: 16 | LR: 2e-05
-----
Overall Intent Accuracy: 93.80%
Overall Slot F1:          98.70%

```

Figure 4 Intent Classification Report (evaluation metrics run)

Intent Classification Report:				
	precision	recall	f1-score	support
addFilter	0.97	0.98	0.98	1367
appreciation	0.82	0.98	0.89	100
checkCapability	0.92	0.97	0.94	101
confirmation	0.88	0.83	0.86	119
deleteFilterValue	0.93	0.86	0.89	176
deleteParameter	0.96	0.90	0.93	104
disconfirmation	0.77	0.91	0.84	101
fallback	0.89	0.77	0.82	100
farewell	0.99	0.95	0.97	100
greeting	1.00	0.88	0.94	100
noMoreFilters	0.92	0.77	0.84	100
recipeRequest	0.95	0.97	0.96	113
requestRecommendation	0.89	0.92	0.91	100
restart	0.93	0.91	0.92	100
accuracy			0.94	2781
macro avg	0.91	0.90	0.91	2781
weighted avg	0.94	0.94	0.94	2781

Figure 5. BIO Slot Classification Report (evaluation metrics run)

BIO Slot Classification Report:				
	precision	recall	f1-score	support
B-cuisine	0.81	0.96	0.88	194
B-dietaryRestriction	0.89	0.85	0.87	174
B-duration	0.90	0.80	0.85	210
B-easyKeyword	0.80	0.83	0.82	108
B-excludeCuisine	0.97	0.64	0.77	99
B-excludeDietaryRestriction	0.82	0.83	0.83	77
B-excludeIngredient	0.83	0.91	0.87	100
B-excludeMealType	0.86	0.98	0.91	44
B-filterType	0.84	0.86	0.85	97
B-ingredient	0.90	0.93	0.91	294
B-ingredientType	0.90	0.71	0.79	51
B-mealType	0.90	0.83	0.86	140
B-recipe	0.94	0.92	0.93	111
B-tag	1.00	0.67	0.80	27
I-cuisine	0.76	0.89	0.82	18
I-dietaryRestriction	0.90	0.82	0.86	254
I-duration	0.92	0.87	0.89	190
I-easyKeyword	0.87	0.75	0.80	87
I-excludeCuisine	0.00	0.00	0.00	7
I-excludeDietaryRestriction	0.84	0.91	0.87	134
I-excludeIngredient	0.93	0.97	0.95	79
I-excludeMealType	0.96	1.00	0.98	22
I-filterType	0.50	0.62	0.56	16
I-ingredient	0.90	0.95	0.92	232
I-ingredientType	0.00	0.00	0.00	5
I-mealType	0.90	0.96	0.93	55
I-recipe	0.94	0.99	0.96	428
I-tag	1.00	0.25	0.40	4
0	1.00	1.00	1.00	41239
accuracy			0.99	44496
macro avg	0.82	0.78	0.79	44496
weighted avg	0.99	0.99	0.99	44496

Figure 6. Merged Slot Type Classification Report (evaluation metrics run)

Merged Slot Type Classification Report:				
	precision	recall	f1-score	support
cuisine	0.80	0.95	0.87	212
dietaryRestriction	0.90	0.84	0.87	428
duration	0.92	0.84	0.88	400
easyKeyword	0.83	0.80	0.82	195
excludeCuisine	0.98	0.60	0.75	106
excludeDietaryRestriction	0.83	0.88	0.86	211
excludeIngredient	0.87	0.94	0.90	179
excludeMealType	0.89	0.98	0.94	66
filterType	0.80	0.84	0.82	113
ingredient	0.91	0.95	0.93	526
ingredientType	0.90	0.64	0.75	56
mealType	0.90	0.87	0.88	195
recipe	0.95	0.98	0.96	539
tag	1.00	0.61	0.76	31
micro avg	0.89	0.89	0.89	3257
macro avg	0.89	0.84	0.86	3257
weighted avg	0.90	0.89	0.89	3257

Figure 7&8. Screenshot of recipe_confirmation.html website with youtube API video implementation


CookingBroski
Confirm or ask for another



Selected recipe

45

4

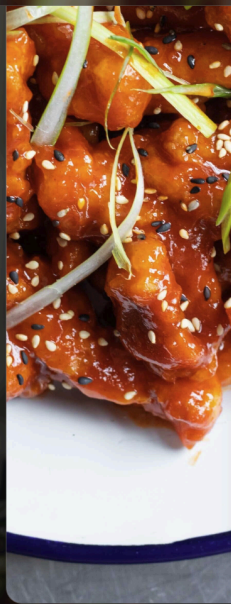
sweet and spicy gochujang fried chicken

Ingredients

- 4 tbsps gochujang
- 6 tbsps ketchup
- 6 tbsps maple syrup
- 4 tbsps soy sauce
- 2 tbsps rice wine vinegar
- 1 tsp brown sugar
- 2 tsp sesame oil
- 50ml water
- 2cm knob of ginger
- 4 cloves of garlic
- 800g skinless boneless chicken thigh fillets
- 2 eggs
- 120g corn flour
- 40g plain flour
- 1 tsp baking soda
- 2 tps garlic powder
- 2 tps onion salt
- 1/2 spring onion
- 1 tsp toasted white/black sesame seeds
- 4 tbsps mayo
- salt
- pepper
- 1l vegetable oil

Instructions

1. add all the ingredients for the sauces into a bowl.
2. grate in the ginger and garlic and mix well.
3. slice your chicken into long strips and place in a large mixing bowl.
4. whisk the two eggs and set aside.
5. to make 2 bowls of flour mix, add 60g corn flour, 20g plain flour, 1/2 tsp baking soda to each mixing bowl.
6. season both bowls of flour with a pinch of salt and pepper and a tsp of garlic powder and onion salt.
7. now add the first bowl of flour mix to your chicken thigh strips and mix well.
8. pour the whisked eggs into the bowl of chicken.
9. then, one at a time, dredge a single chicken thigh strip into the second bowl of flour mix.
10. once they've all been coated, set aside.
11. heat the oil into a deep wok or pot until it reaches 180c.
12. fry the chicken in small batches (making sure to shake off any excess flour before placing in the oil) until they float to the top, this will take a few mins.
13. then place them on a paper-lined rack lined as this will help absorb any excess oil.
14. you want the chicken to be super



14. you want the chicken to be super crispy so pop them back into the oil for a few mins again for the second round of deep-frying.
15. set aside your chicken in a deep bowl.
16. on medium heat, pour the sauce into a separate pot and cook for just a few mins until the sauce thickens a little.
17. pour most of the sauce (set aside two tablespoons) onto the chicken and toss well.
18. mix the remaining 2 tbsps of sauce with some mayo.
19. garnish the chicken with toasted sesame seeds and sliced spring onions and enjoy with your sweet and spicy gochujang mayo.

Watch how to make it



Use the microphone to confirm (*yes*) or ask for another option (*no*).